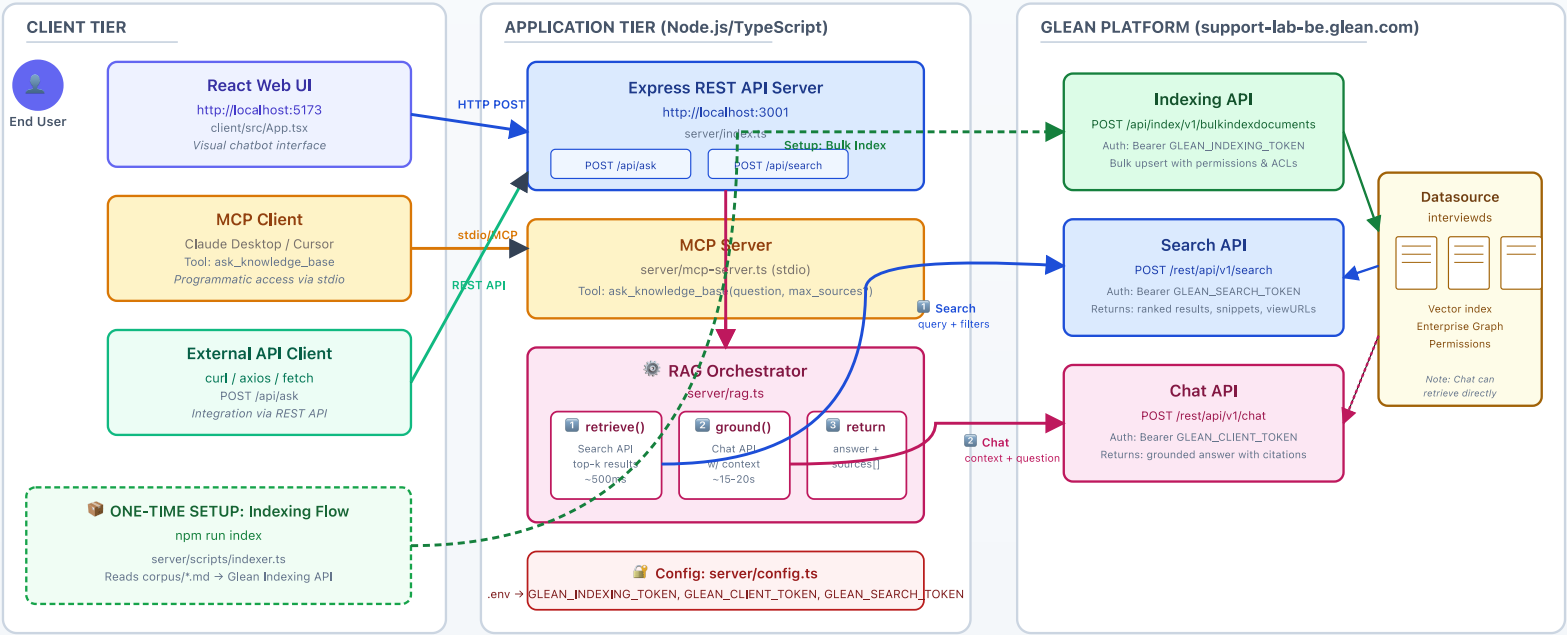


Glean RAG Chatbot — Architecture & Data Flow (Node.js/TypeScript)

Three integration paths: Web UI, REST API, and MCP Tool | Three Glean APIs: Indexing, Search, Chat



Flow Details & Performance Metrics

Flow 1: Web UI (Primary User Path)

- User asks question in React web interface
- POST /api/ask → Express server (localhost:3001)
- RAG orchestrator: retrieve() → ground() → return
- Response: {answer, sources[], meta[latencyMs]}

⌚ Total latency: 15-25 seconds (p95)

Flow 2: MCP Tool (Claude Desktop / Cursor)

- User invokes ask_knowledge_base tool
- MCP server receives request via stdio
- Same RAG orchestrator (shared code path)
- Returns structured JSON to MCP client

⌚ Same latency: 15-25 seconds

Flow 3: External REST API Integration

- External app: curl/axios POST /api/ask
- Express validates request, calls RAG
- Optional: POST /api/search (search-only, no chat)
- Use for: Slack bots, Discord, internal tools

⌚ Search-only: <1 second

Setup Flow: One-Time Indexing (npm run index)

- Read all .md files from corpus/ directory
- Generate stable IDs via content hash (idempotent)
- POST to Glean Indexing API with permissions

⌚ Indexing: ~1 second | Documents searchable in 90 seconds

Performance Breakdown (per question)

Search API: 200-800ms (retrieve top-k results)

Chat API: 15,000-20,000ms (LLM generation)

Total: ~15-25 seconds (dominated by Chat)

First request slower (cold start) | Subsequent requests cached

Key Features

✓ Idempotent indexing | ✓ Citation validation | ✓ Three token separation

Tech Stack

Node.js 18+ | TypeScript 5 | Express | React 18 | Vite | MCP SDK

Authentication

3 tokens: INDEXING (write) | CLIENT (chat) | SEARCH (read)